

VACODER AGENT OS

RUTA 100/100

CONTROL MAESTRO DE CUMPLIMIENTO

De plataforma local Supreme a SaaS cloud multiusuario, autonomo y comercial

Documento	Checklist de implementacion y certificacion
Version	1.0 - 17 de agosto de 2026
Estado base	Supreme 1 + Supreme 2 validados
Objetivo	Coder Enterprise Certification 100/100

REGLA PRINCIPAL

Ningun bloque se marca COMPLETO si typecheck, build, scanner, smoke tests o rollback fallan.

Uso recomendado: imprimir o completar digitalmente marcando [x] en cada criterio.

1. Estado actual verificado

La base local ya esta estable. Esta pagina separa lo que esta comprobado de lo que aun debe convertirse en capacidad SaaS/cloud.

Componente	Estado	Evidencia actual	Impacto
Scanner estructural	VALIDADO	10/10 - 141/141 - 0 fallas - 0 warnings	Base estable
Supreme 1	VALIDADO	Typecheck + build + /supreme	Product score, memoria, modulos, deploy base
Supreme 2 Runtime local	VALIDADO	13/13 paginas + /runtime + APIs start/stop/preview	Runtime local funcional
Runtime cloud	PENDIENTE	Aun depende de procesos locales	Critico para SaaS
Multiusuario SaaS	PENDIENTE	Sin Auth/organizaciones/RLS central	Critico para producto vivo
Preview publica cloud	PENDIENTE	Preview actual localhost	Critico para acceso remoto
Billing/creditos	PENDIENTE	No implementado	Necesario para monetizacion
Enterprise Certification	PENDIENTE	Scanner nuevo aun no creado	Gate final 100/100

Nota: Supreme 1 y Supreme 2 son capacidades base. No sustituyen los bloques Live/Cloud que aparecen mas adelante.

Puntuacion comercial de referencia

Situacion	Score	Meta inmediata	Meta final
Supreme local validado	78/100	82/100 con SaaS Core	100/100 certificacion interna

Checklist previo al Bloque 0

- [x] Supreme 1 instalado y validado
- [x] Supreme 2 instalado y validado
- [x] npm run typecheck pasa
- [x] npm run build pasa
- [x] /runtime responde HTTP 200
- [x] Backup de Supreme 2 identificado
- [] Rama/tag estable de Live creada

2. Reglas de ejecucion por bloques

Gate tecnico: typecheck PASS + build PASS + scanner PASS + smoke tests PASS + rollback PASS.

Gate funcional: La funcion debe probarse desde UI/API; no basta con que exista el archivo.

Gate de persistencia: Cuando aplique, cerrar y reabrir no puede perder estado.

Gate de seguridad: No se permiten secretos en codigo ni acceso entre organizaciones.

Gate de cloud: Una capacidad cloud no se considera terminada si depende de la PC local.

Gate de negocio: Billing, limites y costos deben estar medidos antes de abrir a usuarios.

Formato de cierre obligatorio

- ☐ Instalacion ejecutada
- ☐ Typecheck PASS
- ☐ Build PASS
- ☐ Scanner del bloque PASS
- ☐ Smoke tests PASS
- ☐ Rollback probado
- ☐ Commit creado
- ☐ Evidencia guardada
- ☐ Bloque marcado COMPLETO

Estados permitidos

Estado	Significado	Se puede avanzar?
NO INICIADO	No se ha instalado	No
EN CURSO	Instalado parcialmente	No
BLOQUEADO	Hay error/gate pendiente	No
COMPLETO	Todos los gates pasan	Si

3.1 Bloque 0

BLOQUE 0 - Freeze / baseline estable

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	78/100	78/100	_____

Objetivo: Proteger la version actual antes de abrir la linea Live.

Checklist de implementacion

☐ Crear rama supreme-live	☐ Crear tag coder-supreme2-stable
☐ Guardar reporte scanner 10/10	☐ Respaldar .env.example sin secretos
☐ Registrar hash/commit base	☐ Crear scanner-supreme-live.ps1

Gate de salida

☐ Branch/tag existen

☐ Build y typecheck siguen limpios

☐ Rollback a baseline probado

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.2 Bloque 1

BLOQUE 1 - LIVE SaaS Core

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	78/100	82/100	_____

Objetivo: Convertir Coder en plataforma multiusuario con identidad, organizaciones y proyectos.

Checklist de implementacion

☐ Configurar Supabase Auth	☐ Crear profiles
☐ Crear organizations	☐ Crear organization_members
☐ Crear projects/project_members	☐ Implementar RLS
☐ Crear login/signup	☐ Crear dashboard SaaS
☐ Crear roles OWNER/ADMIN/DEVELOPER/REVIEWER/VIEWER	☐ Separar datos por organizacion

Gate de salida

☐ Dos usuarios no pueden leer proyectos ajenos

☐ Sesion persiste al recargar

☐ RLS tests PASS

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.3 Bloque 2

BLOQUE 2 - Control Plane persistente

Estado	Score entrada	Score salida	Responsable
[] NO INICIADO	82/100	84/100	_____

Objetivo: Eliminar dependencia de rutas C:\... como identidad del proyecto.

Checklist de implementacion

[] Asignar project_id universal	[] Guardar repo/branch/stack/framework
[] Guardar runtime/preview/deploy	[] Guardar ultimo build/score
[] Migrar UI a seleccion de proyecto	[] Persistir memoria y decisiones

Gate de salida

[] Cerrar navegador y reabrir conserva proyecto

[] No se requiere ruta local como identificador principal

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.4 Bloque 3

BLOQUE 3 - Job Orchestrator

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	84/100	86/100	_____

Objetivo: Mover tareas largas fuera del request HTTP y hacerlas reanudables.

Checklist de implementacion

☐ Crear queue	☐ Crear jobs y tasks
☐ Implementar retries	☐ Implementar timeout
☐ Implementar cancelacion	☐ Implementar prioridad
☐ Persistir progreso	☐ Estados queued/planning/running/...

Gate de salida

☐ Cerrar pestaña no cancela tarea

☐ Job reanuda/consulta estado

☐ Retry probado

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.5 Bloque 4

BLOQUE 4 - Cloud Runtime real

Estado	Score entrada	Score salida	Responsable
[] NO INICIADO	86/100	90/100	_____

Objetivo: Ejecutar cada proyecto en entorno aislado fuera de la PC local.

Checklist de implementacion

[] Definir RuntimeProvider	[] Implementar DockerRuntimeProvider
[] Crear worker cloud	[] git clone en workspace
[] npm install/build/dev en container	[] Limites CPU/RAM/disco
[] Timeout y network policy	[] Aislamiento entre workspaces

Gate de salida

[] Proyecto corre con PC local apagada

[] Dos workspaces no comparten filesystem

[] Limites de recursos probados

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.6 Bloque 5

BLOQUE 5 - Realtime Engine

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	90/100	91/100	_____

Objetivo: Hacer que Coder se sienta vivo con eventos y terminal en tiempo real.

Checklist de implementacion

☐ SSE para eventos	☐ WebSocket para terminal
☐ Persistir runtime logs	☐ Persistir build/deploy logs
☐ Reconectar automaticamente	☐ Cursor/offset de eventos
☐ Timeline en UI	

Gate de salida

☐ Recarga de pagina recupera estado

☐ Logs no se pierden

☐ Terminal interactiva funciona

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.7 Bloque 6

BLOQUE 6 - Preview Cloud publica

Estado	Score entrada	Score salida	Responsable
[] NO INICIADO	91/100	92/100	_____

Objetivo: Dar URL HTTPS accesible desde cualquier red para cada workspace.

Checklist de implementacion

[] Hostname por runtime	[] Reverse proxy
[] HTTPS	[] Health checks
[] Expiracion de previews	[] Preview desktop/tablet/mobile
[] Proteccion de preview privada	

Gate de salida

[] Preview abre desde telefono en otra red

[] Health check detecta caida

[] HTTPS valido

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.8 Bloque 7

BLOQUE 7 - GitHub Engine

Estado	Score entrada	Score salida	Responsable
[] NO INICIADO	92/100	93/100	_____

Objetivo: Convertir Git local en workflow real de repositorios, ramas y PR.

Checklist de implementacion

[] Conectar GitHub OAuth/App	[] Importar repo
[] Crear repo	[] Clone/pull/push
[] Crear branch	[] Commit firmado/logico
[] Crear PR	[] Merge/rollback

Gate de salida

[] Coder crea branch y PR desde UI

[] Diff coincide con commit

[] Rollback por commit funciona

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.9 Bloque 8

BLOQUE 8 - Playwright QA Supreme

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	93/100	95/100	_____

Objetivo: Probar la app como usuario real en navegador.

Checklist de implementacion

☐ Instalar Playwright en workers	☐ Chrome desktop
☐ Viewport mobile	☐ Capturar console errors
☐ Capturar network errors	☐ Probar rutas criticas
☐ Probar formularios/botones	☐ Screenshots
☐ Generar QA score	

Gate de salida

☐ App con fallo visual no puede marcar ready

☐ Screenshots guardados

☐ Console/network test PASS

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.10 Bloque 9

BLOQUE 9 - Autonomous Repair Loop

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	95/100	96/100	_____

Objetivo: Reparar automáticamente build/QA hasta alcanzar gates o agotar intentos.

Checklist de implementación

☐ Capturar error estructurado	☐ Localizar archivo/linea
☐ Generar patch	☐ Aplicar patch
☐ Repetir typecheck/build	☐ Repetir Playwright
☐ Max 5 intentos	☐ Rollback final si no resuelve

Gate de salida

☐ Proyecto roto deliberadamente se repara

☐ Loop termina limpiamente

☐ Rollback probado al exceder intentos

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.11 Bloque 10

BLOQUE 10 - Multi-Agent Supreme

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	96/100	97/100	_____

Objetivo: Convertir los roles conceptuales en agentes que ejecutan y coordinan trabajo.

Checklist de implementacion

☐ CEO Agent	☐ CTO Agent
☐ Planner Agent	☐ Builder Agent
☐ Designer Agent	☐ QA Agent
☐ Security Agent	☐ DevOps Agent
☐ Registrar input/output/cost/duration	☐ Orquestar dependencias

Gate de salida

☐ Prompt grande se divide en subtareass

☐ Agentes entregan artifacts

☐ QA/DevOps bloquean release cuando corresponde

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.12 Bloque 11

BLOQUE 11 - Module Store real

Estado	Score entrada	Score salida	Responsable
[] NO INICIADO	97/100	98/100	_____

Objetivo: Instalar modulos funcionales, no solo registrarlos.

Checklist de implementacion

[] Manifest por modulo	[] installer.ts
[] validator.ts	[] rollback.ts
[] tests	[] Auth module
[] Payments module	[] Admin module
[] PDF/Email/CRM modules	[] Migraciones DB

Gate de salida

[] Instalar modulo produce app compilable

[] Desinstalar/rollback no rompe app

[] Tests del modulo PASS

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.13 Bloque 12

BLOQUE 12 - Deploy Engine

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	98/100	98.5/100	_____

Objetivo: Publicar automaticamente tras gates de calidad.

Checklist de implementacion

☐ Provider interface	☐ Vercel provider
☐ Git push previo	☐ Configurar env vars
☐ Deploy preview/prod	☐ Capturar URL
☐ Health check produccion	☐ Visual QA produccion

Gate de salida

☐ URL publica funcional sin terminal manual

☐ Deploy fallido queda auditado

☐ Produccion pasa QA

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.14 Bloque 13

BLOQUE 13 - Secret Vault + Security Enterprise

Estado	Score entrada	Score salida	Responsable
[] NO INICIADO	98.5/100	99/100	_____

Objetivo: Proteger secretos, organizaciones y acciones sensibles.

Checklist de implementacion

[] Vault cifrado	[] No secretos en repo
[] RLS estricto	[] Permission scopes
[] Rate limiting	[] Audit logs
[] Session controls	[] Security scanner continuo
[] Rotacion/revocacion de secretos	

Gate de salida

[] Secret scan PASS

[] Cross-tenant tests PASS

[] Audit trail completo

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.15 Bloque 14

BLOQUE 14 - Billing + credits

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	99/100	99.3/100	_____

Objetivo: Monetizar y limitar consumo de IA/cloud.

Checklist de implementacion

☐ Planes FREE/PRO/BUSINESS/AGENCY/ENTERPRISE	☐ Subscriptions
☐ Credits	☐ Usage events
☐ Runtime minutes	☐ AI tokens
☐ Storage/deploy limits	☐ Webhooks de pago
☐ Invoices/receipts	☐ Bloqueo por limite

Gate de salida

☐ Usuario sin credits es bloqueado correctamente

☐ Uso se contabiliza

☐ Webhook idempotente

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.16 Bloque 15

BLOQUE 15 - Observability

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	99.3/100	99.5/100	_____

Objetivo: Medir salud técnica, éxito de agentes y costos.

Checklist de implementación

☐ Error tracking	☐ Tracing
☐ Metrics	☐ Central logs
☐ Agent success rate	☐ Build success rate
☐ Deploy success rate	☐ AI cost/project
☐ Cloud cost/user	☐ Revenue/margin dashboard

Gate de salida

☐ Alertas funcionan

☐ Dashboard refleja datos reales

☐ Costos reconciliables

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.17 Bloque 16

BLOQUE 16 - Template Marketplace

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	99.5/100	99.7/100	_____

Objetivo: Crear negocios completos desde templates especializados.

Checklist de implementacion

☐ Migracion	☐ Financiera
☐ Inmobiliaria	☐ Courier
☐ Transporte	☐ Marketplace
☐ CRM	☐ Educacion
☐ E-commerce	☐ SaaS
☐ Versionado de templates	

Gate de salida

☐ Template crea proyecto deployable

☐ Incluye auth/roles/db/admin/tests

☐ Version upgrade probado

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.18 Bloque 17

BLOQUE 17 - CEO Mode

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	99.7/100	99.9/100	_____

Objetivo: Evaluar una app como negocio y ejecutar mejoras por prioridad.

Checklist de implementacion

☐ Technical score	☐ Security score
☐ UX score	☐ Monetization score
☐ SEO score	☐ Legal score
☐ Operations score	☐ Boton CORREGIR TODO
☐ Plan de lanzamiento	☐ Readiness report

Gate de salida

☐ Score se basa en evidencia

☐ Corrector ejecuta acciones reales

☐ Readiness cambia tras mejoras

Evidencia / commit / URL / reporte: _____

Observaciones: _____

3.19 Bloque 18

BLOQUE 18 - Enterprise Certification 100/100

Estado	Score entrada	Score salida	Responsable
☐ NO INICIADO	99.9/100	100/100	_____

Objetivo: Crear el gate definitivo independiente del scanner estructural original.

Checklist de implementacion

☐ Architecture audit	☐ Cloud audit
☐ Auth/RLS audit	☐ Runtime/isolation audit
☐ Realtime audit	☐ GitHub/deploy audit
☐ Playwright/repair audit	☐ Multi-agent audit
☐ Security/billing audit	☐ Observability/marketplace audit
☐ Disaster recovery audit	☐ Performance/UX audit

Gate de salida

☐ Todas las categorias 100

☐ 0 fallas criticas

☐ 0 warnings sin justificar

☐ Disaster recovery probado

☐ Reporte final firmado/archivado

Evidencia / commit / URL / reporte: _____

Observaciones: _____

4. Mapa de puntuacion y avance

Bloque	Nombre	Entrada	Salida	Estado
0	Freeze / baseline estable	78	78	[]
1	LIVE SaaS Core	78	82	[]
2	Control Plane persistente	82	84	[]
3	Job Orchestrator	84	86	[]
4	Cloud Runtime real	86	90	[]
5	Realtime Engine	90	91	[]
6	Preview Cloud publica	91	92	[]
7	GitHub Engine	92	93	[]
8	Playwright QA Supreme	93	95	[]
9	Autonomous Repair Loop	95	96	[]
10	Multi-Agent Supreme	96	97	[]
11	Module Store real	97	98	[]
12	Deploy Engine	98	98.5	[]
13	Secret Vault + Security Enterprise	98.5	99	[]
14	Billing + credits	99	99.3	[]
15	Observability	99.3	99.5	[]
16	Template Marketplace	99.5	99.7	[]
17	CEO Mode	99.7	99.9	[]
18	Enterprise Certification 100/100	99.9	100	[]

Progreso global: _____ bloques completos de 19 | **Score actual:** _____ / 100 | **Fecha:** ____/____/____

5. Registro de cierres y evidencias

Bloque	Fecha	Commit/Tag	Scanner	Smoke/QA	Firma/OK
0	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
1	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
2	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
3	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
4	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
5	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
6	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
7	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
8	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
9	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
10	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
11	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
12	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
13	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
14	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
15	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
16	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
17	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____
18	___/___/___	_____	PASS / FAIL	PASS / FAIL	_____

Incidencias bloqueantes

Fecha	Bloque	Incidencia	Resolucion

6. Certificacion final Coder Enterprise 100/100

Esta pagina solo se completa cuando los Bloques 0-18 estan marcados COMPLETO y el scanner Enterprise independiente pasa sin fallas criticas.

Categoria	Score	PASS
Architecture	___ / 100	[]
Cloud	___ / 100	[]
Auth / RLS	___ / 100	[]
Orchestrator	___ / 100	[]
Runtime / Isolation	___ / 100	[]
Realtime	___ / 100	[]
Preview	___ / 100	[]
GitHub	___ / 100	[]
Visual QA	___ / 100	[]
Repair Loop	___ / 100	[]
Multi-Agent	___ / 100	[]
Module Store	___ / 100	[]
Deploy	___ / 100	[]
Security	___ / 100	[]
Billing	___ / 100	[]
Observability	___ / 100	[]
Marketplace	___ / 100	[]
CEO Mode	___ / 100	[]
Disaster Recovery	___ / 100	[]
Performance / UX	___ / 100	[]

RESULTADO FINAL: _____ / 100

[] CERTIFICADO [] NO CERTIFICADO

Fecha final: _____ **Version:** _____ **Commit:** _____

Observaciones finales: _____
